Multi-Agent Competition in SoccerTwos: Reward Shaping, Self-Play, and Imitation Learning

Bo Feng
Georgia Institute of Technology
bfeng66@gatech.edu

Frank F. Yang
Georgia Institute of Technology
frank.yang@gatech.edu

Abstract: We train PPO agents for the 2-vs-2 SoccerTwos environment, in which the default reward (a goal scored or conceded) is too sparse for the policy to gain traction from random initialisation. We add a six-signal reward-shaping wrapper (ball-approach, kick, offensive/defensive, time, team-separation), separate the policy and value networks, and mix the opponent pool 70% self-play / 30% CEIA baseline. The submitted agent reaches **90% (90/100) against the CEIA baseline** and **99% (99/100) against a random opponent**. A third agent warm-starts PPO from a behavioral-cloning policy that reaches 75.7% expert-action accuracy; it does not outperform random initialisation when fine-tuned against CEIA, which we trace to distribution shift, a cold-start critic, and the amplification of small action errors in adversarial play.

Keywords: Multi-Agent RL, PPO, Self-Play, Reward Shaping, Behavioral Cloning

1 Overview

SoccerTwos [1] is a 2-vs-2 physics-based soccer environment built on Unity ML-Agents in which goal events are sparse and the opponent is non-stationary under self-play. We submit three trained agents that share a PPO [2] backbone with Generalized Advantage Estimation [3] but differ in how the environment, opponent distribution, and initialization are handled: (i) **Agent 1 – PPO Self-Play (no shaping)**: a baseline trained with only the default game reward; (ii) **Agent 2 – PPO + Reward Shaping**, which is also our final submission, adds six dense rewards and trains with a 70% self-play / 30% CEIA opponent mixture using a separated policy/value network; (iii) **Agent 3 – BC + PPO Fine-tune**, which collects expert trajectories from an intermediate Agent 2 checkpoint, pretrains by behavioral cloning [4], and then fine-tunes against the CEIA baseline. All training uses Ray RLlib [5] v1.4.0 on the Georgia Tech PACE cluster.

2 Method

2.1 Algorithm and Architecture

Each agent is trained with PPO using the clipped surrogate objective

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \tag{1}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ and $\hat{A}_t$ is computed by GAE with $\lambda = 0.95$. The observation is the default 336-dimensional vector [1] (ray casts plus relative positions/velocities of ball, goals, and the agent itself); the action is a MULTIDISCRETE(3,3,3) tuple controlling move/rotate/kick. Agents 1 and 3 use the Ray RLlib default `vf_share=True` (shared 256×256 MLP for policy and value); Agent 2 uses `vf_share=False` (two independent 256×256 MLPs). With a shared trunk we observed the value loss dominating the policy gradient mid-training and the win rate plateauing; separating the networks was important for reaching our highest win rates.

2.2 Reward Shaping (Environment Modification)

The default game reward ($+1$ on a goal scored, -1 on a goal conceded) is too sparse to train from scratch. We wrap the multi-agent environment with a `RewardShapingWrapper` (`utils.py`) that adds six dense per-step rewards (Table 1). The hypothesis is that providing gradient signal *before* any goals are scored teaches basic ball-interaction skills (approach + kick), then offensive/defensive shaping orients the policy toward goal-directed play, and the separation reward discourages both teammates from clustering around the ball. All weights are kept small (max $|w| = 10^{-3}$) so that the original ± 1 goal signal still dominates as the policy improves; this design follows the principle that auxiliary shaping should not overwhelm the true return [6].

Table 1: Per-step shaped reward signals added to the sparse goal reward.

Signal	Weight	Description
Approach	$+0.0002$	Velocity component toward the ball.
Kick	$+0.001$	Ball acceleration when the agent is within 1.5 units of the ball.
Offensive	$+0.0004$	Ball velocity toward the opponent goal.
Defensive	-0.001	Ball within 5 units of own goal.
Time penalty	-0.00002	Per-step cost to discourage stalling.
Separation	$+0.0001$	Distance between the two teammates (encourage spatial spread).

2.3 Training Strategy and Opponent Composition

We compare three opponent compositions: (a) pure self-play [7] with a 3-snapshot rolling archive (plus the current trainee) that rotates whenever the team's reward exceeds 0.3; (b) team-vs-CEIA training where opponents are sampled predominantly from the fixed CEIA baseline with an opponent-update cooldown to prevent destabilising the policy; and (c) self-play 70% + CEIA 30%, inspired by Brandão et al. [8] who report high win rates in robotic soccer when self-play dominates the opponent pool but a fixed reference opponent is retained. This 70/30 mix, together with the separated policy/value networks (Sec. 2.1), gave us the highest CEIA win rate by combining strategy diversity from self-play with direct exposure to the evaluation opponent. Each iteration samples 20 000 environment steps across 7 parallel workers; training uses a learning-rate schedule from 3×10^{-4} down to 3×10^{-5}, an entropy bonus of 0.005, and gradient clipping at 0.5 (Table 2).

Table 2: Hyperparameters for the final reward-shaped self-play run (Agent 2).

Parameter	Value	Parameter	Value
`learning_rate`	$3 \times 10^{-4} \rightarrow 3 \times 10^{-5}$	`train_batch_size`	20 000
`clip_param`	0.2	`sgd_minibatch_size`	2 048
`entropy_coeff`	0.005	`num_sgd_iter`	10
λ (`GAE`)	0.95	`vf_loss_coeff`	0.5
`grad_clip`	0.5	`vf_share`	False (separated nets)
opponent mix	70% self-play / 30% CEIA	total steps	$\approx$ 30M

2.4 Behavioral Cloning + PPO Fine-tune (Agent 3)

Training PPO from scratch against the full-strength CEIA opponent fails: the random initial policy is too weak to score, so the gradient signal collapses. We tried behavioral cloning as a warm start. (1) **Expert collection**: roll out our best-so-far checkpoint ($\approx 80\%$ win rate vs. CEIA) for 500 episodes, recording roughly 100 k (s, a) pairs. (2) **Supervised pretraining**: train a network with the same architecture as the PPO policy by cross-entropy on the three sub-actions (50 epochs, Adam, cosine learning-rate, batch 2 048; train/val split 90/10). (3) **PPO fine-tune**: inject the BC weights into a fresh PPO trainer via a custom `BCInitCallback`, then fine-tune against 100% CEIA with

a lower learning rate (5×10^{-5}) and a higher entropy bonus (0.01) to compensate for the overly deterministic BC policy.

3 Experimental Results

Training curves. Figure 1 shows episode reward versus environment steps for the self-play baseline (Agent 1) and the reward-shaped self-play run (Agent 2). The reward-shaped curve plateaus much earlier in training (around 1 M vs. 3-4 M environment steps for the unshaped run). We caution that absolute reward magnitudes are not directly comparable between the two curves: the shaping run's episode total folds in the per-step shaped signal, so the y-axis values measure different quantities. The figure supports a qualitative claim about earlier convergence rather than a precise speedup factor.

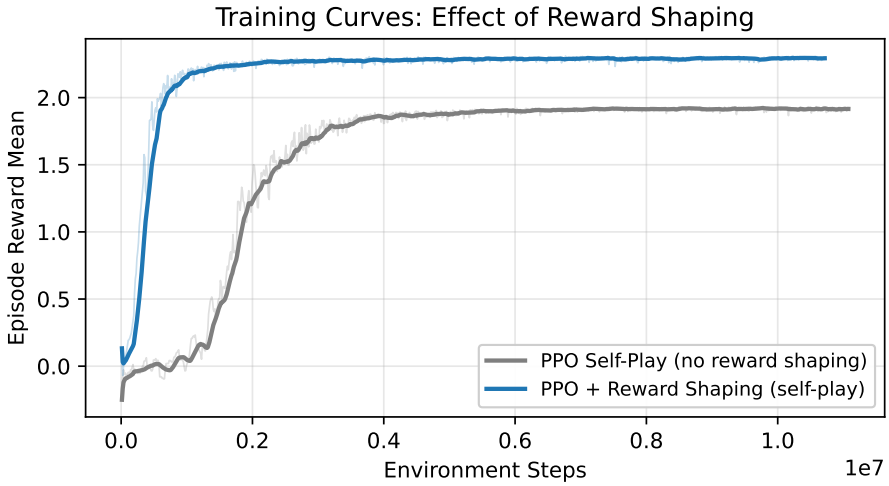

Figure 1: Episode reward vs. environment steps for the self-play baseline (Agent 1) and PPO with reward shaping (Agent 2). Light lines are raw values, bold lines are a 15-iteration moving average. The reward-shaped run plateaus substantially earlier; absolute reward magnitudes are not directly comparable because the shaping run's episode total includes the per-step shaped reward.

Performance comparison. Table 3 summarises win rates against the random and CEIA opponents. Both win rates were measured over 100 matches each (`scripts/eval_vs_ceia.py` and `scripts/eval_vs_random.py`) on the PACE cluster. The submitted agent (*PPO Reward Shaped + Self-Play 70% / CEIA 30%*, checkpoint-2300) reaches **90/100 = 90%** against CEIA and **99/100 = 99%** against the random opponent – both above the project's 9-of-10 target. The pure self-play agent's win rate against CEIA was highly unstable across iterations, illustrating the difficulty of self-play without grounding in a fixed reference opponent. The team-vs-CEIA variant reaches 68% but is outperformed by the 70/30 mixture.

Behavioral cloning ablation. The BC pretraining converged to 75.7% top-1 action-prediction accuracy (validation loss 0.579) in 12 minutes. After 12.5 M timesteps of PPO fine-tuning against 100% CEIA the episode reward was 0.078, indistinguishable from a randomly initialized PPO trained against CEIA from scratch (also 0.078 after a matched number of iterations).

4 Analysis and Discussion

Reward shaping accelerates convergence. Figure 1 shows that reward shaping reaches a plateau substantially earlier than the unshaped baseline. Each of the six dense signals provides a non-zero

Table 3: Win rate of each agent (draws excluded from the percentages). Agent 3 never won a match against CEIA; we therefore also report its mean episode reward. Checkpoint indices listed in individual rows are from different training runs with different starting points and are not directly comparable as a step-count axis.

Agent	vs. Random Agent	vs. CEIA Baseline
Agent 1: PPO Self-Play (no shaping)	–	unstable across iterations
Agent 2 variant: PPO Team (heavy CEIA mix)	–	68%
Agent 2 variant: Reward shaping (self-play)	–	85% (checkpoint-5750)
Agent 2: Reward Shaping + 70/30 (*submitted*)	**99/100 = 99%**	**90/100 = 90%**
Agent 3: BC + PPO fine-tune (bonus method)	–	0% (mean reward ≈ 0.078)

gradient even when no goal is scored, which breaks the chicken-and-egg problem of a sparse-reward MARL environment where both teams initially produce random play.

Self-play instability. Pure self-play training was highly unstable, with the win rate against CEIA varying substantially across iterations. Two settings caused this: (1) with `entropy_coeff` $= 0$, we observed policy entropy fall from $\sim$3.0 to $\sim$0.5 within 1 M steps in our TensorBoard logs, locking in narrow strategies that did not generalise beyond the self-play archive; (2) with `grad_clip` disabled, goal-event rewards produced large updates that erased prior learning. Setting `entropy_coeff` $=$ 0.005 and `grad_clip` $=$ 0.5 removed both failure modes. Switching to `vf_share=False` also helped: with a shared trunk the value loss dominated the policy gradient mid-training and the win rate plateaued below the level reached with separated networks.

Opponent composition and network separation jointly drive the CEIA win rate. Two design choices, not one, gave the submitted agent its 90% CEIA win rate. First, the share of CEIA in the opponent pool: pure self-play was unstable, heavy CEIA training reached 68%, and the 70% / 30% self-play/CEIA mix reached 85–90%. Second, separating the policy and value networks (`vf_share=False`): instead of a shared 256$\times$256 MLP feeding both the policy logits and value head (the Ray RLlib default `vf_share=True`), each function gets its own independent 256$\times$256 MLP, with the value loss weighted at `vf_loss_coeff` $= 0.5$. This eliminates the gradient interference described above — with separate trunks the value-loss gradient can no longer pull the policy's representation toward state-value prediction. With the identical 70/30 mix, a run with shared networks (continued from checkpoint-5122) reached 85%, while a run with separated networks (continued from checkpoint-1700) reached 90%. We caution that these two runs also differ in their starting checkpoint, so the comparison is suggestive of a positive effect of network separation rather than a clean ablation. The 70/30 mix retains enough self-play to develop coordinated behaviour (e.g. passing, blocking) without over-fitting to a single opponent, while still anchoring the gradient signal on the evaluation target.

Why behavioral cloning failed in this adversarial setting. 75.7% expert-action accuracy did not translate into any measurable advantage during PPO fine-tune. Three factors explain the gap. (a) *Distribution shift*: the expert is a near-optimal policy whose state visitation differs sharply from a freshly-initialised PPO learner. The BC student is therefore queried during fine-tune on states the expert rarely visited, where its 24.3% residual action error compounds. (b) *Value-function cold-start*: under `vf_share=True`, BC initialises the shared trunk and policy head, but PPO's value head remains randomly initialised, so early advantage estimates are noisy and policy updates are unstable. (c) *Adversarial amplification*: each incorrect action hands the opponent a scoring opportunity, unlike single-agent or cooperative tasks where small errors are recoverable. The pattern matches the DAgger covariate-shift analysis [9]. A follow-up using DAgger or GAIL [10] — which both re-query the expert on states the learner visits — would address (a) directly, though we have not run this experiment.

Code and reproducibility. All training scripts, the reward-shaping wrapper, the BC pipeline, and the evaluation utilities are available at
https://github.com/Bo-Feng-1024/soccer-twos-starter. The reward modification referenced in Table 1 is implemented in `utils.py` (lines 18-102), the BC pipeline in `train_bc_finetune.py` and `collect_expert_data.py`, and the self-play training loop in `train_ray_selfplay.py`.

References

[1] B. Oliveira. A pre-compiled soccer-twos reinforcement learning environment with multi-agent gym-compatible wrappers and human-friendly visualizers. https://github.com/bryanoliveira/soccer-twos-env, 2021.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[4] D. A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.

[5] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica. RLlib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 3053–3062, 2018.

[6] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

[7] T. Bansal, J. Pachocki, S. Sidor, I. Sutskever, and I. Mordatch. Emergent complexity via multi-agent competition. In *International Conference on Learning Representations (ICLR)*, 2018.

[8] B. Brandão, T. W. de Lima, A. Soares, L. Melo, and M. R. O. A. Maximo. Multiagent reinforcement learning for strategic decision making and control in robotic soccer through self-play. *IEEE Access*, 10:72628–72642, 2022. doi:10.1109/ACCESS.2022.3189021.

[9] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011.

[10] J. Ho and S. Ermon. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.